A Model for Entity Profile Management Through Inference in Trails

André Wagner and Jorge Luis Victória Barbosa

Universidade do Vale do Rio dos Sinos (UNISINOS)
Unidade Acadêmica de Pesquisa e Pós-Graduação
Av. Unisinos, 950, São Leopoldo - RS, Brazil

Abstract

To be written...

1 Protótipo e Estudo de Caso

O estudo de caso apresentado é baseado na geração simulada de trilhas de estudantes de um curso de gradução em Ciência da Computação. Foi decidido pela utilização de um cenário simplificado, de modo que o mesmo pudesse ser apresentado de forma aprofundada.

1.1 Definio do Cenrio

O cenário simula um curso de programação em linguagens orientadas a objeto. Deste curso participam três alunos (André, Pedro e Maria), cada um dos quais representado por uma entidade no eProfile.

O curso consiste em um conjunto de módulos relacionados a orientação a objetos. Destes módulos fazem parte conceitos básicos da matéria (classes, herança, *interfaces* e ponteiros) e linguagens de programação orientadas a objeto (C++, Python e Java). O estudante não é obrigado a concluir todos os módulos. No final do curso, no entanto, ele deve escolher a linguagem de programação que ele utilizará para responder as questões da prova final. Dependendo da linguagem que escolher, ele deve ter concluído, no mínimo, os módulos referentes àquela linguagem.

Por exemplo, caso o aluno decida realizar a prova em Java. Neste caso ele deve ter estudado classes, herança e *interfaces*, além do Java propriamente dito. Como a linguagem Java não contém o conceito de ponteiros, ele não é obrigado a ter cursado este módulo.

As obrigatoriedades dos módulos são representados pela Figura 1. No exemplo proposto, para que o estudante possa realizar a prova na linguagem Python, ele deve ter concluído no mínimo os módulos Classes, Herança e Python.

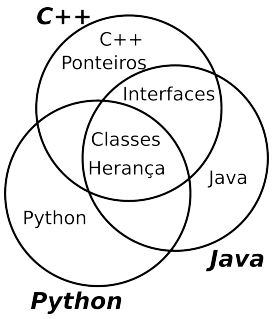

Figura 1: Módulos de estudo obrigatórios

O ambiente é integrado a um sistema de educação ubíqua denominado LOCAL [1], utilizado para a distribuição contextualizada de objetos de aprendizagem (LO – *Learning Objects*). Os conteúdos dos módulos são distrubídos pelo LOCAL como objetos de aprendizagem, e o LOCAL registra em um gerenciador de trilhas cada vez que um módulo é iniciado ou concluído.

Cada evento (início ou conclusão de um objeto de aprendizagem) é registrado contendo as seguintes informações: (i) entidade que realizou o evento, (ii) objeto de aprendizagem, (iii) tipo de evento (início ou conclusão), (iii) data e hora e (iv) se é a primeira vez que o estudante realiza este tipo de evento.

O LOCAL registra no gerenciador de trilha não apenas os eventos, mas também as informações relacionadas àqueles eventos. Por exemplo, toda vez que um evento possui relação com um objeto de aprendizagem que está sendo mencionado pela primeira vez, é necessário que sejam incluídas todas as informações relevantes a este objeto, de modo que o eProfile possa utilizar estas informações ao montar a infe-

rência.

Levando em conta o ambiente apresentado, a questão que o LOCAL deseja que o eProfile processe e registre no perfil da entidade é: esta entidade é proeficiente em determinada linguagem? As subsessões a seguir descrevem como ocorre este processo de inferência.

1.2 Ontologia

À medida que o registro das informações vai sendo realizada pelo LOCAL no gerenciador de trilhas, o eProfile vai solicitando informações atualizadas ao gerenciador de trilhas. A forma como o LOCAL e o gerenciador de trilhas se comunicam não é relevante ao eProfile, desde que o gerenciador de trilhas envie as informações em um formato que o eProfile seja capaz de interpretar.

De acordo com a Figura **??**, o gerenciador de trilhas envia as informações ao módulo conversor do eProfile, que as converte para o formato OWL. As informações enviadas são de dois tipos: (i) as ocorrências, isto é, a trilha propriamente dita e (ii) as informações acessórias, que ajudarão o eProfile a montar a ontologia necessária para a inferência das informações. No cenário que está sendo descrito, um módulo concluído é um exemplo de ocorrência, enquanto as informações de dependências entre os módulos de aprendizagem são informações acessórias.

Tendo por base as informações enviadas pelo gerenciador de trilhas de acordo com o cenário descrito na Sessão 1.1, o eProfile montou a ontologia mostrada na Figura 2. Cada círculo representa uma classe da ontologia.

É importante notar que cada uma das classes pai tem alguma representação na ontologia de trilha do eProfile, apresentada na Figura **??**.

Algumas classes possuem informações que são necessárias para que o eProfile possa determinar se um estudante é proeficiente em determinada linguagem. Um exemplo disto são os módulos referentes a conceitos básicos de orientação a objetos, que estão representados na ontologia como classes filhas da classe `ObjectOrientation`. A classe `OO-Interface`, por exemplo, diz respeito a um módulo que estuda o conceito de *interfaces*. Este conceito está presente nas linguagens C++ e Java, mas não em Python. Assim, esta classe é descrita da seguinte forma em OWL[1]:

```
1 Class: <el:OO-Interfaces>
2     SubClassOf:
3         <el:ObjectOrientation>,
4         <el:relatesTo> some <el:C++>,
5         <el:relatesTo> some <el:Java>
```

No exemplo, a linha 1 registra a classe, indicando que ela é filha da classe ObjectOrientation (linha 3, ver Figura 2). Além disso, as linhas 4 e 5 utilizam a propriedade `relatesTo` para indicar que o conceito de *interfaces* está relacionado às linguagens de programação (e, por conseqüência, às classes) `C++` e `Java`. Note-se que não é feita relação à classe `Python`.

Os eventos de requisição e conclusão do estudo de um objeto de aprendizado são registrados como classes filhas da classes `RequestedLOEvent` e `FinishedLOEvent` respectivamente que, por sua vez, são filhas da classe `LearningEvent`. Cada especialização da classe exige informações adicionais, de modo que a classe `FinishedLOEvent`. Este conjunto de classes é representado da seguinte forma:

```
1  Class: <el:Event>
2      SubClassOf:
3          Thing,
4          <el:hasEntity> some <el:Entity>,
5          <el:hasDateTime> exactly 1 xsd:dateTime,
6  Class: <el:LearningEvent>
7      SubClassOf:
8          <el:Event>,
9          <el:hasLearningObject> some
10                                 <el:LearningObject>
11 Class: <el:FinishedLOEvent>
12     SubClassOf:
13         <el:LearningEvent>,
14         <el:isFirstTime> exactly 1 xsd:boolean
```

Desta forma, cada evento registrado deve ser vinculado a uma entidade (linha 4) e ter uma data e hora (linha 5); um evento de aprendizagem deve ter um objeto de aprendizagem relacionado (linhas 9 e 10); e um evento de término de aprendizado deve ter registrado se o evento ocorreu pela primeira vez.

O código abaixo representa o registro de um evento, onde a entidade André (linha 4) concluiu um objeto de aprendizagem sobre Classes (linha 5) pela primeira vez (linha 6) em 08/07/2012 às 10h30 (linha 7). Note-se que, como esta classe é filha da classe `FinishedLOEvent`, as propriedades requisitadas na classe estão sendo informadas nas classes filhas.

```
1 Class: <el:FinishedLOEvent1>
2     SubClassOf:
3         <el:FinishedLOEvent>,
4         <el:hasEntity> some <el:Andre>,
5         <el:hasLearningObject> some <el:OO-Classes>
6         <el:isFirstTime> value true,
7         <el:hasDateTime> value "2012-07-08T10:30:00"
```

1.3 Regras

A aplicação de regras é discutida na Seção **??**.

Para adicionar ao perifl de um aluno se ele é proeficiente em uma determinada linguagem (que é o objetivo do cenário, de acordo com a Seção 1.1, os seguintes passos devem ser realizados:

[1] Embora o eProfile utilize internamente a sintaxe OWL/RDF [3], a sintaxe Manchester-OWL está sendo usada neste artigo por ser mais compacta a amigável [2].

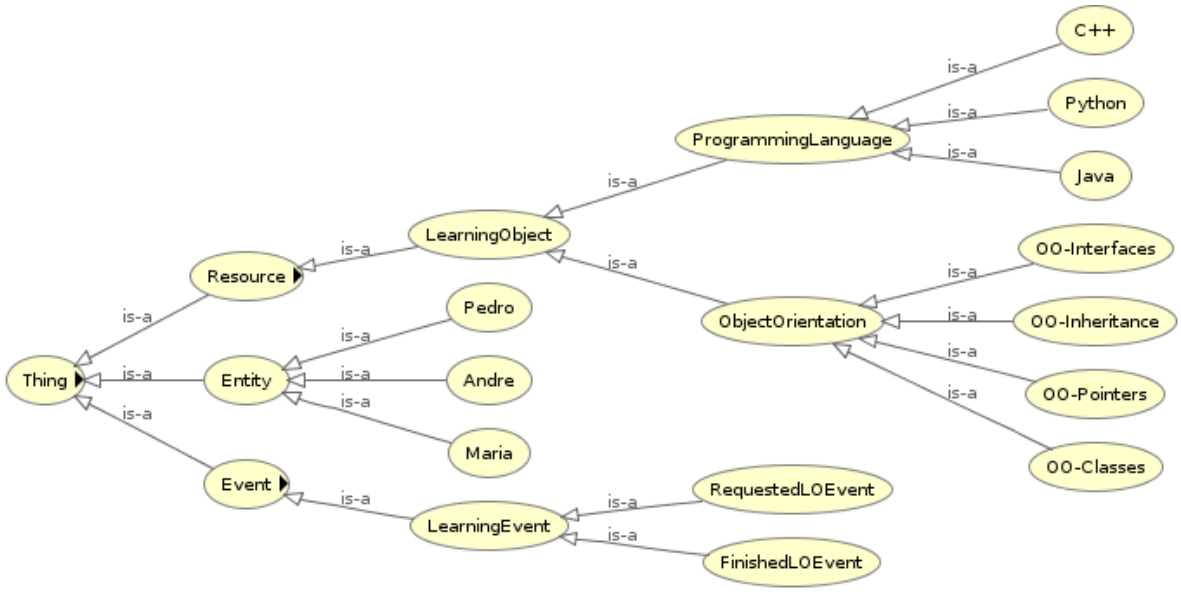

Figura 2: Ontologia do cenário apresentado

1. Identificar os objetos de aprendizados requeridos para a proeficiência na linguagem;
2. Identificar os objetos de aprendizagem concluídos pelo estudante referentes ao item 1;
3. Verificar se todos os objetos de aprendizagem necessários foram concluídos.

Neste exemplo, será verificado se o estudante é proeficiente na linguagem Java. Para tal, ele deve ter concluído os objetos de aprendizagens que fazem relação com a linguagem Java (neste caso, classes, herança em *interfaces*), bem como o objeto de aprendizado da linguagem Java. Na ontologia, isto refere-se à criação de uma nova classe (`JavaRequirement`) que seja equivalente à classe `Java` (que é filha da classe de `LearningObject` e, portanto, um objeto de aprendizado, ou quaisquer outras classes cuja propriedade `relatesTo` aponte para a classe `Java`. Assim, esta regra fica definida da seguinte forma:

```
Class: <el:JavaRequirement>
    EquivalentTo:
        <el:Java>
         or (<el:relatesTo> some <el:Java>)
```

O segundo passo é a identificação dos objetos de aprendizagem concluídos pelo estudante relacionados ao Java. Para tal, é necessário que:

1. os eventos sejam do tipo `FinishedLOEvent` (só é importante saber os objetos de aprendizado que o estudante concluiu, e não os que ele iniciou);
2. os eventos tenham sido realizados pela entidade específica;
3. os eventos sejam relacionados ao Java, ou seja, filhas da classe `JavaRequirement` criada acima;
4. os eventos tenham sido realizados pela primeira vez (somente é importante saber os objetos de aprendizado concluídos pelo aluno, e não quantas vezes).

Para que o eProfile possa identificar estes eventos para a entidade Andre, a seguinte regra é enviada ao eProfile:

```
Class: <el:JavaLessonsTaken>
    EquivalentTo:
        <el:FinishedLOEvent>
         and (<el:hasEntity> some <el:Andre>)
         and (<el:hasLearningObject> some
                                <el:JavaRequirement>)
         and (<el:isFirstTime> value true)
```

Uma vez que estas duas regras estejam criadas na ontologia, o eProfile irá executar o raciocinador, que gerará uma nova ontologia OWL, com as classes inferidas identificadas como filhas das classes criadas.

A partir da execução do raciocinador, uma regra de atualização de perfil é criada, onde são comparados o número de módulos Java com o número de objetos de aprendizagem relacionados ao Java que foram concluídos pela entidade. Se o número for igual, o usuário é proeficiente. Esta regra é definida da seguinte forma:

```
JavaProficience <=
  (Count(SubClasses(JavaRequirement))) equals
          (Count(SubClasses(JavaLessonsTaken)))
```

2 Conclusão

Referências

[1] J. Barbosa, R. Hahn, S. Rabello, and D. Barbosa. Local: a model geared towards ubiquitous learning. *SIGCSE Bull.*, 40:432–436, March 2008.

[2] W3C. OWL 2 web ontology language manchester syntax, 2009. `http://www.w3.org/TR/owl2-manchester-syntax/`. Acessado em 08/07/2012.

[3] W3C. OWL web ontology language guide, 2011. `http://www.w3.org/TR/owl-guide/`. Acessado em 05/05/2012.